

分层自改进：面向任务特定可进化智能体挂载器的框架

周泰林 香港科技大学
tzhouaq@connect.ust.hk

摘要

现代大语言模型智能体通常通过手动修改提示、工具或工作流来改进，而模型周围的可执行脚手架——挂载器——在部署后通常被视为固定工件。本文研究另一种方案：挂载器是任务特定的且可持续进化；每个任务族维护自己的挂载器，通过固定的任务注入接缝在迭代间热切换，并利用环境反馈进行重写。我们提出分层自改进（Hierarchical Self-Improvement, HSI）框架，其中单个冻结的大语言模型 M 在三个层级作用域上运行：执行任务的任务挂载器 H 、进行重写的进化器 H ，以及在冻结外层锚点下重写进化器策略代码的元进化器。思考开/关设计通过在任务执行期间禁用推理、在自修改期间启用推理，隔离了挂载器进化的贡献。HSI 受两个因素约束：反馈保真度界限，因为进化需要信息丰富的奖励信号来指导选择；以及骨干能力界限，因为挂载器重新设计无法克服冻结模型的局限性。在 BALROG 上以 DeepSeek-V4-Flash-Preview 作为冻结骨干，HSI 在中等难度任务上相比初始挂载器取得了一致的提升（BabyAI 上 +39.3，Crafter 上 +33.0，TextWorld 上 +25.0，MiniHack 上 +15.0，均为原始进度百分比），同时在 BabaIsAI 子套件上获得了强大的留出泛化能力（从 20% 未见过数据划分中，BreakStop 最佳测试 0.98，GoTo 1.00）。在超出骨干能力的任务（NLE）上，挂载器进化没有带来改进。这些结果表明，任务特定挂载器进化是在明确经验极限下改进冻结大语言模型智能体的可行途径。代码见 <https://github.com/TailinZhou/hsi>。

1 引言

围绕大语言模型（Large Language Model）的可执行脚手架（即“挂载框架”，包括提示词、工具编排、记忆与验证逻辑）已成为智能体性能的关键决定因素，即便在相同模型基座下，不同挂载框架设计也会产生显著差距 [李等人, 2026b, 姚等人, 2026]。然而，两个根本性挑战仍未解决。首先，现有自我改进方法尚未完全内生地触及挂载框架层。哥德尔式自我改进系统主要演化智能体的逐步决策代码 [尹等人, 2025, 张等人, 2026b, 王等人, 2025, 翁等人, 2026, 张等人, 2026c]，而针对更广泛脚手架的挂载框架工程方法往往依赖外部提议者或更强的设计模型 [李等人, 2026b, 张等人, 2026a, 林等人, 2026a]。其次，目前尚不清楚所观察到的挂载框架演化收益究竟反映真实能力提升，还是仅体现测试时搜索，因为近期评估揭示了显著的过拟合与有限的泛化 [王等人, 2026b]。这些局限性引出一个核心问题：当底层模型被冻结时，一个智能体能否

内生地演化自身框架以提升性能，以及最终限制这种提升的因素是什么？

我们通过分层自我改进（Hierarchical Self-Improvement, HSI）框架来解决这一问题，该框架使单个冻结的大语言模型 M 能够通过其自身框架的分层演化而非参数更新来实现改进。HSI 在三个层次范围内运作：任务框架 H ，围绕 M 执行任务；演化器，在迭代中重写 H ；以及元演化器，在更高一层重写演化器的策略代码。为防止无限制的自我引用，元演化器自身的执行逻辑保持冻结作为外部锚点，将自我修改限制为分层且经经验验证的编辑。思考开/关设计隔离了框架演化的贡献：任务执行期间禁用推理以固定模型的逐步能力上限，重写期间启用推理以最大化成功自我修改的机会。由此产生的循环包括五个阶段：种子选择、主演化、提交选择、元演化和终端最佳版本选择。

我们在 BALROG（Paglieri 等人，2025 年）上实例化 HSI，这是一个长时程基于文本的交互式游戏基准，呈现出自然的难度梯度。使用 DeepSeek-V4-Flash-Preview 作为冻结的主干模型，我们在两种互补协议下评估了引导进化：全量集进化以衡量可实现的分布内改进，以及分割进化以评估留出泛化能力。在包括 TextWorld、Crafter 和 BabyAI 在内的中等难度环境中，HSI 在保持主干模型和推理配置不变的情况下，始终优于匹配的初始引导基线。在较简单的 BabaIsAI 子套件上，进化后的引导能够泛化到未见任务，而在 NLE 上，由于主干模型提供的能力和反馈信号不足，引导进化没有产生有意义的改进。这些结果揭示了冻结 LLM 智能体内生引导进化的潜力和局限性。

HSI 实例化了一个通用模板，即任务特定且持续可演化的任务框架：每个任务族维护自己的任务框架，可通过固定的任务注入接缝在迭代间热插拔，并利用环境反馈进行精炼。关键设计原则是冻结锚点下的分层演化：任务框架变化时重写过程保持锚定，而重写过程本身可以在更高一层演化，同时外层执行锚点保持固定。通过在所有作用域中保持相同的冻结 M ，HSI 实现了内生任务框架演化，而无需无限制的自我引用。该范式受两个基本限制约束：反馈保真度界限，因为演化需要信息丰富的奖励信号来指导选择；以及骨干能力界限，因为任务框架重新设计无法克服底层模型的局限性。我们的贡献是：

1. **分层自我改进（HSI）框架。**我们引入了一个分层自我改进框架，使冻结的 LLM 能够通过嵌套重写作用域演化自己的任务框架，同时通过冻结的外层锚点防止无限制的自我引用。
2. **受控模型上限下的积极证据。**在 BALROG 上使用冻结的 DeepSeek-V4-Flash 骨干，我们展示了任务特定任务框架演化在中等难度环境中取得一致增益，并泛化到留出的 BabaIsAI 子套件。无思考任务执行协议隔离了推理时间推理作为混杂因素。
3. **缩放极限的经验表征。**本文识别了驾驭演化的两个实用边界：反馈可用性与骨干能力。在不同难度级别的环境中，这些边界刻画了内生驾驭演化何时能够改进冻结的大语言模型智能体，以及何时进一步重新设计带来的收益有限。

2 相关工作

HSI 涉及三个研究领域：哥德尔式自我改进、面向大语言模型智能体的驾驭工程，以及自我改进系统的评估与理论刻画。本文回顾这些方向，并阐明 HSI 与现有方法的区别。

2.1 哥德尔式自我改进

哥德尔机（Gödel Machine）[施米德胡贝，2003] 提出了自改进系统的基础思想，该系统能够修改自身程序，包括负责未来修改的程序。

条件，前提是此类更改能够被验证为可提升性能。近期研究已通过运行时自编辑、进化搜索和基于群体的探索，将这一思想应用于大语言模型智能体。哥德尔智能体（Gödel Agent）[尹等人，2025] 通过运行时代码修改实现自指改进，而达尔文哥德尔机（Darwin Gödel Machine, DGM）[张等人，2026b]、赫胥黎-哥德尔机（Huxley-Gödel Machine, HGM）[王等人，2025] 和群体进化智能体（Group-Evolving Agents, GEA）[翁等人，2026] 则探索了日益广泛的进化机制。超智能体（HyperAgents）[张等人，2026c] 进一步扩展了这一方向，使元级机制也可被编辑。然而，这些方法主要将可进化边界置于智能体的决策过程或程序执行过程，而更广泛的智能体框架能否内生进化仍是一个开放问题。

2.2 线束工程与在线自适应

框架工程研究围绕大语言模型的可执行组件（包括提示、工具、记忆和验证机制）如何塑造智能体行为与性能。元框架（Meta-Harness）[李等人，2026b] 将框架优化形式化为使用更强提议模型的外层搜索问题，而自动框架（AutoHarness）[楼等人，2026] 探索了通过基于搜索的优化实现自动框架综合。近期研究关注内生框架适应，即智能体通过反馈和验证修改自身脚手架。自框架（Self-Harness）[张等人，2026a] 研究通过弱点挖掘和基于回归的接受机制实现自生成框架精化，而智能体框架工程（Agentic Harness Engineering, AHE）[林等人，2026a] 将可观测性和组件级反馈识别为有效自我改进的关键因素。框架 X（HarnessX）[陈等人，2026b] 进一步将框架形式化为可组合的自适应系统，框架锻造（HarnessForge）[陈等人，2026a] 则研究框架与策略的联合进化。

另一条并行研究线考虑部署期间的框架适应。测试时框架进化（TTHE）[聂等人，2026] 在测试时利用执行迹进化框架，而实时软件工程智能体（Live-SWE-Agent）[夏等人，2025]、持续框架（Continual Harness）[卡滕等人，2026] 和自适应自动框架（Adaptive Auto-Harness）[刘等人，2026] 研究在线和持续适应。尽管取得了这些进展，现有方法大多聚焦于进化任务框架本身，而非进化支配框架发现、选择和重写机制的元机制。

2.3 评估、归因与理论极限

近期研究凸显了评估自我改进智能体以及将性能提升归因于真实能力增长的难度。Harness-Bench [姚等人，2026 年] 将框架设计确立为独立的评估维度，表明在相同模型下，不同框架配置会导致显著的性能差异。框架更新并非框架收益 [林等人，2026 年 b] 进一步区分了智能体生成框架更新的能力与其从这些更新中获益的能力。重新思考框架演化的评估 [王等人，2026 年 b] 表明，自动框架演化可能无法超越简单的测试时扩展基线，并且可能遭受严重的过拟合。

从理论视角出发，王等人 [2026a] 刻画了自我改进智能体的统计极限，并证明当可达假设族具有有界复杂度时，分布无关的 PAC 保证在自我修改下得以保持。这为理解自我改进系统的能力边界提供了理论视角。

2.4 与先前工作的比较

本文不重新综述现有方法，而是围绕启发 HSI 的三个设计问题组织比较。

可演化的边界究竟在哪里？ 哥德尔式自我改进方法 [尹等人，2025，张等人，2026b，王等人，2025，翁等人，2026，张等人，2026c] 主要演化智能体的决策过程或执行代码。框架工程方法 [李等人，2026b，张等人，2026a，林等人，2026a] 针对更广泛的脚手架，但通常依赖外部优化信号或更强的提议模型。HSI 研究了一个更广泛的内生可编辑表面：协调提示、工具、记忆、状态以及跨步骤交互模式的框架，而执行任务的同一冻结模型也执行演化。

框架演化应如何扩展？ 先前框架优化中的一个常见假设是，单一改进的框架可以跨任务泛化。然而，近期评估研究揭示，演化后的框架可能过拟合，并且无法超越简单的测试时扩展基线 [Wang 等人, 2026b]。HSI 转而采用任务特定的演化范式，其中每个任务族维护其自身的演化框架，并通过留出任务划分来评估泛化能力。因此，扩展轴并非一次性优化的通用框架，而是在受控评估下任务特定框架的持续演化。

在自我修改过程中，什么应保持不变？ 先前分析认为，自改进系统需要对可编辑表面和外部评估机制施加显式约束 [翁, 2026, 王等人, 2026a]。HSI 采用分层约束：任务挂载器通过固定 `using_harness` 接口可热插拔，进化器策略在更高层级可编辑，元进化器在冻结的外层锚点下运行。此外，评估信号和数据划分保持在智能体控制之外。该设计为研究内生挂载器进化提供了经验框架，同时保持对自我修改的明确边界。

3 分层自我改进

本文提出 HSI，一个框架，其中单个冻结的大语言模型通过对其挂载器及进化该挂载器的过程进行分层自我修改来提升任务性能。HSI 通过分层架构将不断演化的面向任务的脚手架与进化机制本身分离。本文首先介绍定义该设置的设计原则（§ 3.1），然后呈现整体架构和进化循环（§ 3.2），最后详细说明各阶段及其结构不变量（§ 3.3）。

3.1 设计原则

在 HSI 中，挂载器是任务特定的且持续可进化的：每个任务族维护自己的挂载器，该挂载器通过固定的任务注入接缝在迭代间进行热插拔，并利用环境反馈进行精炼。这种设计自然需要分层分离。单层设计会将挂载器的面向任务行为与负责重写它的策略耦合在一起，使得优化对象与优化器本身不可分离。分层分离则允许任务挂载器进化，同时其重写过程保持锚定，并允许重写过程在更上一层进化，同时保留冻结的外部锚点。这将自我修改局部化到结构化和经验验证的编辑上，而非无限制的自我引用。

以下两条原则定义了在大语言模型 M 下如何实现分层自我修改。在本文中，底层模型参数保持固定；改进仅来自对挂载器及管理其进化的过程的修改。

原则 1（单个冻结模型，三个挂载器作用域）。 HSI 使用单一冻结的大语言模型 M 在三个层级范围内运行。在任务执行范围， M 中，执行任务执行器 H 与环境交互。在进化器范围， M 中，通过种子选择、执行器进化和候选选择，在迭代过程中修改 H 。在元进化器范围， M 中，修改进化器策略本身，包括种子生成、提交选择、存档维护和最终版本导出等决策。

所有三个作用域共享相同的冻结模型 M 、提示格式和 `react()` 原语。它们仅在可用工具和执行上下文上有所不同。作用域之间通过显式内存边界分隔：任务挂载器、进化器和元进化器交互维护独立历史，而非代表独立智能体。与外部提议者方法 [张等人, 2026a, 林等人, 2026a, 李等人, 2026b] 不同，HSI 使用相同的冻结模型进行执行和进化。

原则 2（自主决定的探索-利用）。 HSI 在进化过程中不规定显式的探索-利用调度。该框架不决定智能体何时应检查代码、评估候选、提交更改、记录经验，或在探索与利用之间分配精力。相反，这些决策被视为由 M 控制的可进化策略的一部分。

该框架仅提供原子交互原语、进化反馈信号和结构不变量。这些包括评估奖励、已提交版本谱系以及存储在 `BOOTSTRAP.md` 中的持久经验。智能体负责确定如何将信号

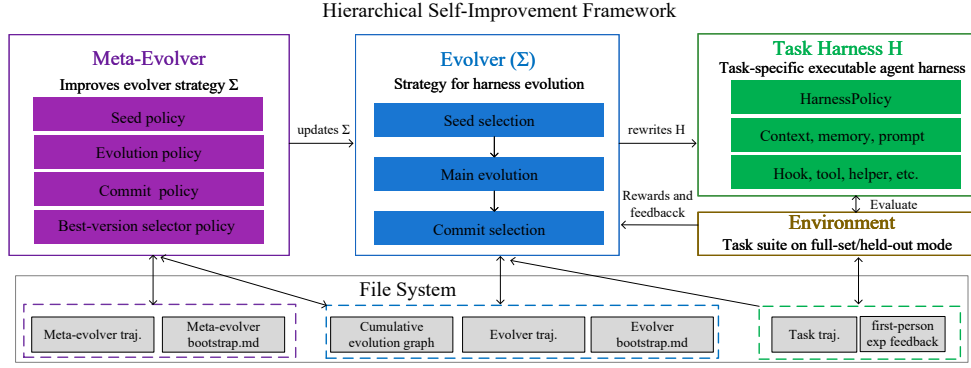


图 1: HSI 框架。单一冻结的大语言模型 M 在三个层级范围内运行，具有不相交的可编辑表面。任务执行范围执行任务特定的执行器 H ；进化器范围通过种子选择、主进化和提交选择重写 H ；元进化器范围重写进化器策略 Σ ，包括种子、进化、提交和最终版本选择策略。

组合成有效的进化策略，而框架仅约束此类适应发生的边界。

3.2 层级架构与自治智能体循环

上述设计原则定义了一种架构，其中单个冻结的大语言模型 M 在多个层级作用域中运行，并具有不同的可编辑表面。核心思想是递归但有界的自我修改：任务框架 H 可以演化，演化过程 H （即演化器策略 Σ ）本身也可以演化，而控制 Σ 的外部执行逻辑保持冻结。这种分离使得分层自我改进能够同时优化对象和优化过程，而不会引入不受限制的自我引用。

三个层级作用域。分层自我改进由同一冻结模型 M 操作的三个作用域组成（图 1）。任务框架作用域执行当前任务框架 H ，其中包含智能体面向任务的组件，包括提示、工具、记忆、状态管理、钩子以及决定模型如何与环境交互的策略。框架是演化过程中的主要可编辑对象，并通过固定的注入接缝与任务连接。

演化器作用域在迭代过程中修改任务框架。它执行由 Σ 定义的演化过程，该过程决定了候选框架的生成、评估和选择方式。每次迭代包括三个阶段：种子选择、主演化和提交选择。演化器通过框架评估接收环境反馈，并通过演化档案和引导记忆维护持续的演化信息。

元演化器作用域在框架演化过程之上运行。它修改 Σ 本身，包括负责种子生成、候选演化、提交选择和最终版本选择的策略。与任务框架和演化器策略不同，执行元演化的执行逻辑不可编辑。它从不可变的初始化模板加载，并作为限制自我修改的外部冻结锚点。

自治演化循环。三个作用域共享相同的冻结模型 M 、提示格式和推理原语，但保持独立的执行上下文和记忆边界。完整的演化过程形成一个循环，包括四个迭代阶段：种子选择、主演化、提交选择和元演化。前三个阶段修改任务框架 H ，而第四个阶段修改演化策略 Σ 。经过 T 次迭代后，最终的版本选择阶段导出最终框架以供评估。

与预定义的优化流程不同，分层自改进框架（HSI）并未在这些阶段内指定明确的探索计划。同一个冻结模型根据可用的工具和反馈信号，决定如何检查、修改、评估和保留候选版本。该框架仅提供

结构性约束，包括可编辑边界、评估接口和冻结的外部锚点。这种设计使得框架及其优化策略能够内生演化，同时保持受控的自我改进边界。

3.3 阶段机制

演化过程包含五个阶段。前三个阶段作用于任务框架 H ，第四个阶段修改演化策略 Σ ，最后一个阶段选择导出的框架进行评估。每个阶段实现为一个有界的 `react()` 循环，其核心决策委托给冻结的大语言模型 M ，同时明确约束可编辑表面和结构不变量。实现细节见附录 A。

3.3.1 种子选择：假设生成

设 H_t 表示迭代 t 开始时的框架，设 $\mathcal{G}_t = (V_t, E_t)$ 表示累积演化图。每个节点 $v \in V_t$ 对应一个已提交的框架快照，并标注奖励 r_v ，而边存储描述版本之间关系的语义信息。

种子选择从 $\mathcal{G}_t$ 中选择一个祖先，并为下一次演化迭代生成结构化假设：选择（

$$(\hat{H}_t, h_t) = \text{seed_} \quad \mathcal{G}_t, M). \quad (1)$$

该决策由 M 根据先前奖励、演化历史和积累的经验做出。生成的假设包含四个要素：选定的锚定版本、选择它的动机、预期的改进方向以及一个可证伪标准。该假设被注入后续演化过程，将演化从无约束的变异转变为由明确预测引导的目标导向搜索。

3.3.2 主要演化：可热插拔的测试框架重写

主要演化阶段中， M 直接修改任务测试框架 H 。给定种子测试框架 $\hat{H}_t$ 和假设 h_t ，演化器生成候选版本：

$$\{V_t^{(k)}\}_{k=1}^{K_t} = \text{main_evolution}(\hat{H}_t, h_t; M). \quad (2)$$

可编辑范围包括测试框架的所有面向任务的组件，如提示、工具、记忆、状态管理、钩子和执行策略。候选修改通过任务-测试框架作用域进行评估，该作用域返回奖励反馈以指导后续编辑。

演化过程本身不受固定优化计划约束。相反， M 决定何时检查代码、提出修改、评估候选版本以及终止迭代。所有重写中唯一保持不变的约束是任务注入接口：虽然内部测试框架组件可能改变，但连接任务与测试框架的外部接口保持固定。这一不变性使得演化版本能够直接比较，并使测试框架在迭代间可热插拔。

3.3.3 提交选择：多提交池

主要演化之后，提交选择阶段选择一组候选版本以供未来探索：

$$C_t = \text{commit_selection}(\{V_t^{(k)}\}_{k=1}^{K_t}, \mathcal{G}_t; M). \quad (3)$$

HSI 并非仅选择奖励最高的候选版本，而是维护一个包含多个演化方向的多样化提交池。每个选中的版本连同 M 生成的语义理由一起添加到演化图中，使未来的种子选择能够对成功、失败和未探索的分支进行推理。

3.3.4 元演化：演化演化器

元进化将进化过程向上提升一个层级，通过修改控制骨架进化的策略 Σ ：

$$\Sigma_{t+1} = \text{meta_evolution}(\mathcal{G}_t \cup C_t, \Sigma_t; M). \quad (4)$$

Σ 的可编辑表面包括负责种子选择、主进化、提交选择和最终版本选择的程序。通过修改 Σ ，元进化器不仅改变候选骨架，还改变用于发现未来骨架的搜索策略。

元进化器被限制在进化策略空间内。其自身的执行逻辑保持不变，并从外部初始化模板加载。这个冻结的锚点防止递归自修改变得无界，同时保留适应进化过程本身的能力。

3.3.5 最佳版本选择：泛化优先导出

经过 T 次迭代后，最终阶段选择部署的骨架：

$$H^* = \text{best_version_selection}(\mathcal{G}_T, M, \Sigma). \quad (5)$$

与中间提交选择不同，后者为未来探索保持多样性，此阶段优先考虑泛化。候选版本在验证性能上进行评估，选定的骨架被导出用于留出评估。选择程序本身仍然是 Σ 的一部分，允许元进化适应最终部署决策的制定方式。

4 实验

4.1 实验设置

基准与度量。我们在 BALROG [帕列里等人, 2025 年] 上评估 HSI，这是一个长时程基于文本的交互环境基准，旨在评估大语言模型智能体的规划、记忆、探索 and 工具使用能力。BALROG 包含六个具有不同能力需求的环境：BabyAI、BabaIsAI、Crafter、MiniHack、TextWorld 和 NLE。

BabyAI 和 BabaIsAI 评估结构化环境中的指令遵循和导航能力。Crafter 要求长时程规划、资源管理和顺序决策。TextWorld 评估多步推理和对象操作。MiniHack 和 NLE 提供了日益具有挑战性的类 Rogue 环境，具有复杂的状态空间和稀疏反馈。这些环境共同构成了一个自然的难度谱系，从冻结骨干网络能取得非平凡性能的任务，到性能仍然有限的任务。

BALROG 使用回合级 % 进度 [帕列里等人, 2025 年] 报告性能，该指标在 0 – 100 尺度上衡量任务完成度。我们在进化过程中将该度量重新缩放到 $[0, 1]$ 。候选挂载器使用随机下置信界奖励进行排序：

$$r = \mu - z \frac{\sigma}{\sqrt{n}}, \quad (6)$$

其中 μ 和 σ 分别表示评估试验的均值和标准差， $z = 0.5$ 在我们的实验中。该奖励降低了进化过程中随机高奖励轨迹的影响。所有报告的结果均使用原始进度百分比均值，而非 LCB 奖励。

基准选择的理由。我们选择 BALROG 是因为它为研究长时程交互式任务中的框架进化提供了一个受控环境。与静态评估设置不同，BALROG 要求智能体维护状态、进行多步推理、通过工具与环境交互，并根据执行反馈调整行为。这些特性暴露了框架设计可以影响的组件，包括记忆管理、状态跟踪、探索策略和动作协调。此外，BALROG 在统一的评估框架内包含了跨越不同难度区间的环境。这使我们能够研究框架进化在哪些方面提供了可测量的增益，以及在固定骨干网络上改进仍然有限的方面，从而为自改进智能体的实用边界提供实证视角。

骨干网络与进化配置。所有实验均使用 DeepSeek-V4-Flash（通过 deepseek-v4-flash-preview 应用程序编程接口服务访问）作为冻结骨干 M ，贯穿任务框架、进化器和元进化器范围。每次进化运行包含 $T = 5$ 次外层迭代，每次迭代最多执行 80 个 `react()` 步骤。

为了将框架进化的贡献与推理时推理隔离开来，在任务框架范围内禁用扩展推理，而在进化器和元进化器范围内启用扩展推理。该配置在开发评估、验证评估、最佳版本选择和最终测试中保持不变。因此，任务执行期间观察到的改进不能归因于推理时额外的推理计算。

我们进一步限制进化空间，使得所有任务交互必须通过冻结骨干 M 进行中介。进化可以修改框架组件，包括提示、工具、记忆、状态管理和控制逻辑，但不能用外部搜索过程或非大语言模型策略替换模型。

评估协议。BALROG 环境是程序化生成的，每次 `evaluate()` 调用都会采样一个新的初始种子。我们使用两种评估协议来衡量不同的泛化设置。

分布内进化。在进化和最终评估中使用相同的任务集，而每个评估回合由新采样的环境种子生成。该协议衡量框架进化是否能在先前遇到的任务的随机变化下提升性能。我们将此协议应用于 TextWorld、BabyAI、Crafter、MiniHack 和 NLE。

在进化过程中，每个候选框架每次调用仅用一个回合进行评估，以提高效率。对于最终评估，我们使用完整的回合预算以获得更稳定的性能估计。

留出任务泛化。对于 BabyIsAI，我们基于子套件类别构建任务族划分：BreakStop、GoTo、Make 和 Advanced。每个子套件被划分为开发、验证和测试部分。测试划分在整个进化过程中保持不可访问。

该协议评估进化后的测试框架能否泛化到同一任务族中未见过的任务。我们报告了在 BreakStop、GoTo 和 Make 上的留出结果。高级子套件仅包含三个任务，因此无法提供足够样本进行有意义的留出评估。

基线。我们使用三类参考比较。首先，初始测试框架是在相同骨干网络和评估协议下未经进化评估的原始手工测试框架。这作为衡量测试框架进化效果的主要受控基线。其次，我们与公开报告的 BALROG 排行榜结果进行比较（如有）。这些比较提供了与前沿模型在其原生配置下的背景对比，包括不同的骨干网络和推理设置。第三，我们将 HSI 与外部提议者测试框架优化方法区分开来。依赖更强外部模型来设计或优化测试框架的方法不纳入受控比较，因为它们基于与 HSI 不同的假设，HSI 研究固定骨干网络下的内生进化。

4.2 HSI 性能

我们在 § 4.1 引入的两种协议下评估 HSI。本节首先研究设置 A，其中进化和评估使用相同的任务分布并进行随机重采样，然后研究设置 B 的留出泛化。

4.2.1 设置 A：分布内测试框架进化。

表 1 将 HSI 与 BALROG 排行榜以及使用相同冻结骨干网络的受控初始测试框架基线进行比较。关键比较在于底部三行，其中所有配置均使用 DeepSeek-V4-Flash，仅在是否以及如何进化测试框架上有所不同。

从相同的骨干网络和任务时推理配置出发，HSI 在所有非平凡套件上均显著优于初始测试框架。元进化开启配置在 BabyAI 上提升进度百分比 +39.3，在 Crafter 上提升 +33.0，在 TextWorld 上提升 +25.0，在 MiniHack 上提升 +15.0，同时保持骨干网络和任务时推理预算不变。这些结果表明

表 1：设置 A 下的 BALROG 排行榜比较。顶部区块列出了前沿模型在其原生配置下的公开排行榜数据（检索于 2026 年 8 月 3 日），以进度百分比报告 [帕列里等人，2025年]（评估回合的均值 $\pm$ 标准差）。底部区块报告了使用单一冻结的 DeepSeek-V4-Flash 骨干网络在同一任务套件上的 HSI，隔离了持续可进化的热插拔任务框架的贡献：初始框架基线、元进化关闭臂和元进化开启臂（加粗），后者导出部署的框架。BabaIsAI 被省略，因为我们的子套件协议（§ 4.2.2）与排行榜的混合任务协议不同。平均值报告了五个环境的未加权均值，其标准差为每个环境标准差的未加权均值。

LLM	BabyAI	Crafter	TextWorld	MiniHack	NLE	Avg
Gemini-3-Pro	96.0 $\pm$ 2.8	57.3 $\pm$ 4.4	60.2 $\pm$ 7.5	40.0 $\pm$ 7.7	6.8 $\pm$ 3.2	52.1 $\pm$ 5.1
Gemini-3.1-Pro-Thinking	98.0 $\pm$ 2.0	55.0 $\pm$ 6.4	75.7 $\pm$ 6.4	27.5 $\pm$ 7.1	2.6 $\pm$ 0.3	51.8 $\pm$ 4.4
Gemini-3.1-Pro	100.0 $\pm$ 0.0	46.8 $\pm$ 4.2	66.5 $\pm$ 7.5	35.0 $\pm$ 7.5	3.0 $\pm$ 0.5	50.3 $\pm$ 3.9
Gemini-3-Flash	86.0 $\pm$ 4.9	45.0 $\pm$ 6.3	50.2 $\pm$ 8.1	30.0 $\pm$ 7.2	4.0 $\pm$ 0.8	43.0 $\pm$ 5.5
Grok-4	76.0 $\pm$ 6.0	57.3 $\pm$ 3.9	62.9 $\pm$ 7.9	17.5 $\pm$ 6.0	1.8 $\pm$ 0.8	43.1 $\pm$ 4.9
Claude-Opus-4.5	80.0 $\pm$ 5.7	49.5 $\pm$ 3.1	51.4 $\pm$ 8.4	27.5 $\pm$ 7.1	2.0 $\pm$ 0.5	42.1 $\pm$ 5.0
Claude-Opus-4.5-Thinking	72.0 $\pm$ 6.3	48.6 $\pm$ 3.2	59.0 $\pm$ 8.0	30.0 $\pm$ 7.2	2.4 $\pm$ 0.3	42.4 $\pm$ 5.0
Gemini-2.5-Pro-Exp-03-25	80.0 $\pm$ 5.7	55.0 $\pm$ 6.0	49.2 $\pm$ 8.2	17.5 $\pm$ 6.0	1.7 $\pm$ 0.2	40.7 $\pm$ 5.2
DeepSeek-R1	74.0 $\pm$ 6.2	36.4 $\pm$ 3.8	21.8 $\pm$ 6.1	25.0 $\pm$ 6.8	1.4 $\pm$ 0.5	31.7 $\pm$ 4.7
GPT-5-minimal-think	80.0 $\pm$ 5.7	39.1 $\pm$ 4.1	30.6 $\pm$ 7.0	20.0 $\pm$ 7.3	1.3 $\pm$ 0.5	34.2 $\pm$ 4.9
Claude-3.5-Sonnet	68.0 $\pm$ 6.6	32.7 $\pm$ 3.2	42.1 $\pm$ 5.4	15.0 $\pm$ 5.6	0.6 $\pm$ 0.5	31.7 $\pm$ 4.3
GPT-4o	77.6 $\pm$ 3.7	33.1 $\pm$ 2.3	39.3 $\pm$ 5.2	10.0 $\pm$ 4.7	0.4 $\pm$ 0.4	32.1 $\pm$ 3.3
DS-V4-Flash (Init harness)	42.0 $\pm$ 3.5	11.6 $\pm$ 5.0	40.0 $\pm$ 6.2	0.8 $\pm$ 1.9	0.0	18.9 $\pm$ 3.3
DS-V4-Flash 带 HSI (元进化关闭)	77.3 $\pm$ 1.2	36.4 $\pm$ 1.6	46.0 $\pm$ 2.4	5.8 $\pm$ 3.8	0.0	33.1 $\pm$ 1.8
	81.3 $\pm$ 4.2	44.6 $\pm$ 3.2	65.0 $\pm$ 3.0	15.8 $\pm$ 2.9	0.2 $\pm$ 0.3	41.4 $\pm$ 2.7

一个冻结的大语言模型可以通过对其周围框架的内生修改获得显著的性能提升。

HSI 在与前沿系统原生配置的对比中也达到了有竞争力的性能。在 TextWorld 上，HSI 实现了 65.0 % 的进度，超过了 Grok-4 (62.9)、Claude-Opus-4.5-Thinking (59.0) 和 Gemini-3-Flash (50.2)。在 Crafter 上，HSI 达到了 44.6，优于 DeepSeek-R1 (36.4)、GPT-5-minimal-think (39.1) 和 GPT-4o (33.1)。这些比较作为上下文参考提供，而受控的初始框架比较则隔离了框架进化的效果。

元进化的效果。表 1 中的元进化关闭消融隔离了进化策略本身进化的贡献。移除元进化器降低了每个评估套件上的性能：BabyAI 从 81.3 降至 77.3，Crafter 从 44.6 降至 36.4，TextWorld 从 65.0 降至 46.0，MiniHack 从 15.8 降至 5.8。最大的改进出现在 TextWorld (+19.0) 和 MiniHack (+10.0) 上，这表明随着框架搜索空间变得更加复杂，调整进化过程变得越来越有益。

NLE 没有提供有意义的元进化关闭比较，因为两种配置都获得了接近零的奖励。元进化开启的结果 0.2 表明进化接收到的任务反馈不足，无法发现有用的框架修改。

进化动态与能力边界。在进化运行过程中，最大的性能提升通常发生在第一次迭代，随后在后续迭代中获得较小的增量收益。这种行为表明，早期的测试框架重新设计捕获了主要的改进，而后续迭代则通过探索和选择来完善现有解决方案。

BALROG 上的性能模式也揭示了测试框架进化的一个实用局限性。虽然 HSI 在冻结骨干产生信息反馈的任务上有所改进，但在 NLE 上并没有实质性提升，因为初始能力和奖励信号都极其有限。这一观察结果与 § 3.1 中讨论的反馈和能力约束一致：测试框架进化可以重组模型周围的行为，但无法克服模型无法生成有用交互信号的任务。

表 2: BabaIsAI 子套件在设置 B（20% 留出测试的分割进化）下的结果。“最佳开发”表示进化过程中选择的最高开发奖励。测试结果报告为任务进度的均值 $\pm$ 跨任务标准差。初始测试框架为三次基线运行的平均值。

子套件	初始测试框架	最佳开发	最佳测试（元开启）	最佳测试（元关闭）
BreakStop	0.0333 ± 0.0334	1.0000	0.9800 ± 0.0632	1.0000 ± 0.0000
GoTo	0.1818 ± 0.0802	1.0000	1.0000 ± 0.0000	0.9636 ± 0.0809
Make	0.0000	0.5556	0.3625 ± 0.3284	0.3375 ± 0.2029

4.2.2 设置 B: BabalsAI 子套件上的留出泛化。

为了评估进化任务之外的泛化能力，我们在三个 BabaIsAI 子套件上进行了留出评估：**BreakStop**、GoTo 和 Make。每个子套件进化一个独立的测试框架，同时保持骨干、进化预算、初始化模板和评估协议不变。唯一变化的因素是任务族。所有实验使用相同的冻结 DeepSeek-V4-Flash 骨干，并采用 § 4.1 中描述的任务时无思考配置，并比较元进化开启和元进化关闭变体。

每个子套件遵循设置 B，使用 20% 的留出测试分割。高级子套件被排除，因为它只包含三个任务，不足以进行有意义的分割评估。表 2 总结了结果。

结果显示了两种不同的状态。对于面向导航的任务（BreakStop 和 GoTo），HSI 实现了近乎完美的留出性能：元开启变体分别达到 0.98 和 1.00 的测试进度，而元关闭变体也取得了相当的结果。这些结果表明，进化后的测试框架发现了可复用的交互模式，这些模式可以迁移到观察到的开发任务之外。

相比之下，制作（Make）仍然具有相当大的挑战性。尽管框架进化相较于零样本初始框架有所改进，但保留集上的性能仍然有限（元开启时为 0.36，元关闭时为 0.34）。较小的提升和较大的方差表明，多步骤制作需要的能力超出了进化过程中发现的可复用框架变换。

在 BALROG 环境中，出现了相同的定性模式：在冻结骨干已经表现出有意义能力的任务上，框架进化提供了最大的改进；在接近能力边界的任务上，改进较小且噪声较大；在难以获得有用反馈的任务上，改进有限。这一观察结果与第 3.1 节讨论的两个实际局限性一致：框架进化可以重组和放大现有模型能力，但无法完全克服反馈不足或骨干的根本局限性。

4.2.3 进化轨迹分析。

为了理解自我改进是如何出现的，我们可视化了来自 Crafter（设置 A，图 2）和 BabaIsAI-Make（设置 B，图 3）的代表性进化轨迹。每条轨迹记录了五次迭代中的奖励演变，以及选定的种子、框架修改、提交池和元进化更新。

Crafter 轨迹。Crafter 轨迹展示了典型的分布内进化模式。最初的迭代主要引入缺失的任务表示：暴露潜在奖励信号、结构化库存信息以及改进动作-状态对齐。后续迭代探索更专门的机制，包括基于规则的建议和安全约束。最佳版本出现在第 4 次迭代，之后额外的修改产生了回归，这表明进化并非单调地改进每个分支，而是在非凸的框架设计空间中进行搜索。

元进化器的贡献在于将成功的局部发现转化为可复用的进化启发式规则。在多次迭代中，它用更高层次的原则更新 Σ ，例如优先考虑结构化状态表示而非原始观测，以及在性能平台期附近避免过于激进的探索。这些变化影响未来的搜索行为，而不是直接修改任务性能。

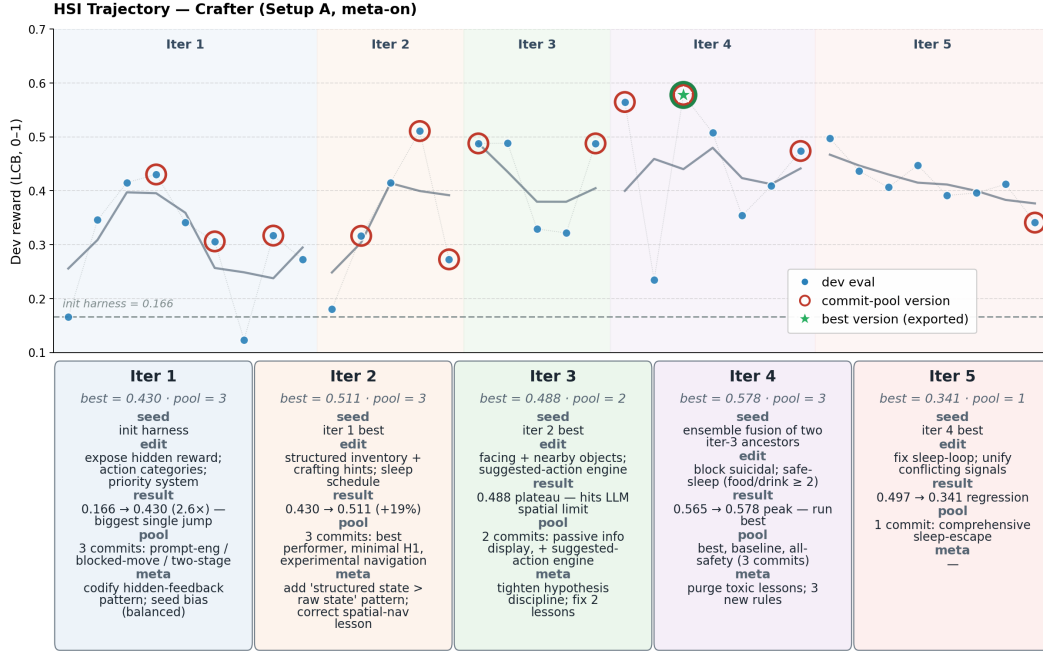


图 2: HSI 在 Crafter 上的演化轨迹（设置 A，元演化开启）。开发奖励从初始线束基线 0.166 攀升至第 4 次迭代的迭代最优 0.578（以绿色星标标记为导出的最佳版本），第 5 次迭代出现回归。每次迭代的标注卡在任务线束和演化器作用域上报告四个字段（种子来源、主演化编辑、结果、提交池），外加一个绿色调元字段，总结元演化器在该次迭代中对演化器策略 Σ 的重写内容。演化器揭示的主导杠杆是使隐藏的游戏反馈显式化：奖励信号、库存状态和制作可行性，依次在线束上下文中暴露；元演化器的贡献是把这些模式编码为 Σ ，以便后续迭代继承。

BabalsAI-Make 轨迹。BabalsAI-Make 轨迹展示了留出演化。与 Crafter 不同，在 Crafter 中评估是在同一任务集的随机重采样下进行的，设置 B 明确暴露验证行为。开发奖励通过连续的线束重新设计得到改善，包括空间抽象、基于 BFS 的规划和 LLM 引导的导航。留出验证标记显示，若干发现的机制能够迁移到开发任务之外，尽管剩余差距表明多步制作仍接近冻结骨干的能力边界。

在两条轨迹中，HSI 表现出共同的演化模式：早期迭代发现缺失的抽象，中期迭代引入结构化算法组件，后期迭代细化或剪除竞争设计。这提供了定性证据，表明自我改进通过渐进式线束重构而非简单的提示优化发生。

5 讨论与结论

5.1 分层自我改进的经验教训

在 BALROG 环境中运行 HSI 揭示了关于内生线束演化何时有效的若干观察。

- 执行反馈是可靠演化的基础。仅通过静态检查难以评估线束修改。有用的信号是修改后的线束在环境中执行时是否产生改进的行为。这一观察推动了 HSI 的设计选择：保持固定的任务注入接口，通过交互而非代码级评估候选版本

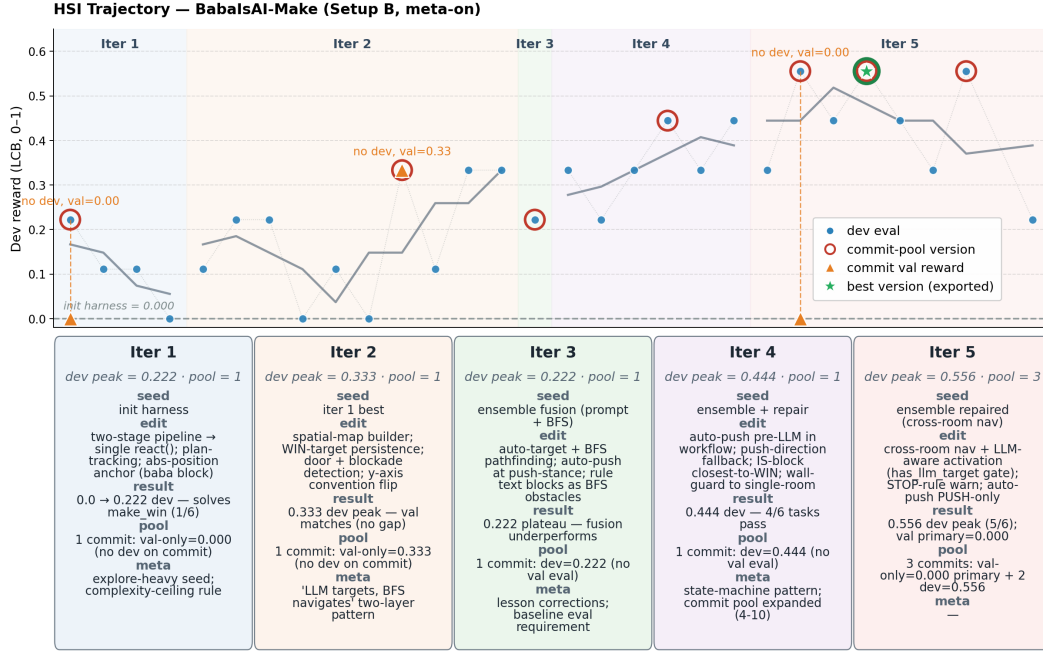


图 3: BabalsAI-Make (设置 B, 元开启) 上的 HSI 演化轨迹。主蓝色曲线为全部 reward_history 次评估的开发奖励; 红色圆环标记提交池版本。橙色三角形 (△) 标记在验证模式下最终确定的提交 (提交本身未记录开发评估): 每个 △ 位于提交的锚点 x 处, y 轴为验证奖励, 虚线垂线从开发锚点降至验证奖励, 使开发 → 验证评估结构可直接从轨迹中读出。开发峰值在五次迭代中攀升 $0.222 \rightarrow 0.333 \rightarrow 0.444 \rightarrow 0.556$, 代理引入了带计划跟踪的单次反应重写 (迭代 1)、带 WIN 目标持久化的空间地图构建器 (迭代 2)、带 BFS 寻路的自动目标计算 (迭代 3)、带方向回退的自动推动机制 (迭代 4) 以及 LLM 感知的跨房间导航 (迭代 5); 元进化器将“LLM 定目标, BFS 导航”的两层模式固化, 并逐步将提交池从一个版本扩展到三个版本。△ 标记揭示了哪些迭代的主要提交在留出验证任务上进行了验证, 这是设置 B 独有的评估结构。

启发式方法, 并利用奖励反馈指导后续修改。缺乏基于执行的反馈, 自我修改很容易产生看似合理的变化, 却无法转化为任务性能的提升。

- 单种子进化提高了归因清晰度。框架进化本质上是随机的: 不同的搜索轨迹可以发现不同的改进。HSI 有意避免基于种群的并行扩展, 而是遵循单一进化谱系, 从而可以将性能变化归因于框架重新设计, 而不是候选吞吐量的增加。这种设计以搜索效率换取对内生改进的更清晰测量。基于种群的探索 [张等人, 2026b, 翁等人, 2026] 仍然是互补的, 可以作为额外的扩展维度纳入。
- 任务特定演化似乎是一个实用的扩展方向。结果表明, 演化出的测试框架捕获的是任务族特定的结构, 而非普遍可迁移的解决方案。在 BabalsAI 中, BreakStop 和 GoTo 上强大的留出性能表明, 演化能够在任务族内发现可复用的策略, 而相同的机制不会自动迁移到差异显著的环境中。这表明, 未来自改进智能体的扩展可能受益于为不同任务分布维护专门的、可演化的测试框架, 而非仅依赖单一通用测试框架。
- 骨干能力决定了可达到的改进前沿。

当环境提供信息丰富的反馈且冻结模型能够有效利用这些信息时，测试框架演化能够提升性能。在我们的实验中，在模型可达到的能力范围内的任务上（TextWorld、BabyAI、Crafter 和 BabaIsAI-GoTo/BreakStop）取得了显著提升，而更困难的环境（MiniHack、BabaIsAI-Make 和 NLE）则表现出较小或可忽略的改进。这些结果表明存在一个经验能力边界：测试框架演化扩展了固定模型的有效性，但无法完全克服模型底层推理能力的局限或环境反馈不足的问题。

5.2 结论

HSI 提出了一种分层自改进框架，其中单个冻结的大语言模型 M 在三个范围内运行：与环境交互的任务测试框架、重写测试框架的演化器，以及重写演化策略本身的元演化器。通过可编辑边界分离这些范围并保留冻结的外部锚点，HSI 实现了递归改进，同时避免了不受限制的自我引用。

在 BALROG 上的实验表明，固定的 DeepSeek-V4-Flash 骨干网络仅通过内源性测试框架演化即可实现显著改进。在分布内评估下，HSI 在多个环境中一致地优于初始测试框架，同时保持骨干网络和任务时推理配置不变。在 BabaIsAI 上的留出评估中，演化出的测试框架能够泛化到同一任务族内的未见任务，表明这些改进不仅限于单条轨迹。

同时，结果揭示了明显的极限。进化需要信息丰富的反馈，而奖励极其稀疏的环境提供的改进信号不足。此外，最终性能仍受底层冻结模型能力的制约。这些观察表明，自我改进的智能体不应被视为替代更强的模型，而应被视为一种机制，通过基于环境的适配器调整，从现有模型中系统性地提取额外能力。

分层自我改进（HSI）提供了这一范式的一个实例：在冻结骨干模型下的每任务、持续可进化的适配器框架。未来的方向包括将分层进化与更强的基础模型、更丰富的反馈环境以及可扩展的基于种群的搜索相结合。

致谢

这项工作作为一个开源研究项目开发。作者感谢开源社区以及使该项目成为可能的模型、环境和工具的开发者的支持。

当前版本代表了对冻结语言模型下分层自我改进的初步探索。由于实际计算限制，评估集中在选定的基准、骨干模型和比较上，而非全面的实证研究。我们将此版本视为持续社区探索的基础，包括在更多模型、环境和自我改进设置上的评估。

完整源代码可在以下地址获取：<https://github.com/TailinZhou/hsi>。我们欢迎研究人员和从业者在此框架基础上进行构建、复现和扩展。

参考文献

- Mingju Chen, Can Lv, Guibin Zhang, Heng Chang, and Shiji Zhou. Harnessforge: Joint harness and policy evolution for adaptive agent systems, 2026a. URL <https://arxiv.org/abs/2606.01779>.
- Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. Harnessx: A composable, adaptive, and evolvable agent harness foundry, 2026b. URL <https://arxiv.org/abs/2606.14249>.

- Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. Sia: Self improving ai with harness & weight updates, 2026. URL <https://arxiv.org/abs/2605.27276>.
- Seth Karten, Joel Zhang, Tersoo Upaa Jr, Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vodrahalli. Continual harness: Online adaptation for self-improving foundation agents, 2026. URL <https://arxiv.org/abs/2605.09998>.
- Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement, 2026a. URL <https://arxiv.org/abs/2607.15524>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026b. URL <https://arxiv.org/abs/2603.28052>.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026a. URL <https://arxiv.org/abs/2604.25850>.
- Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving llm agents, 2026b. URL <https://arxiv.org/abs/2605.30621>.
- Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams, 2026. URL <https://arxiv.org/abs/2606.01770>.
- Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness, 2026. URL <https://arxiv.org/abs/2603.03329>.
- Elias Lumer, Sahil Sen, Kevin Paul, and Vamse Kumar Subbiah. Recursive agent harnesses, 2026. URL <https://arxiv.org/abs/2606.13643>.
- Haochen Luo, Yi Huang, Sichun Luo, Fengyuan Liu, Lei Li, Zefa Hu, Junlan Feng, and Qi Liu. Harness-aware self-evolving: Co-evolving model weights, harness, and task solutions, 2026a. URL <https://arxiv.org/abs/2607.03935>.
- Xiaotian Luo, Fengxingyu Wang, Chuanrui Hu, Dizhan Xue, and Yafeng Deng. Self-evolving agent harnesses via gated semantic quality-diversity, 2026b. URL <https://arxiv.org/abs/2607.13683>.
- Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. Tthe: Test-time harness evolution, 2026. URL <https://arxiv.org/abs/2607.08124>.
- Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. BALROG: Benchmarking agentic LLM and VLM reasoning on games. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=fp6t3F669F>.
- Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent, 2025. URL <https://arxiv.org/abs/2504.15228>.
- Jürgen Schmidhuber. Gödel machines: Fully self-referential universal self-improvers, 2003. URL <https://arxiv.org/abs/cs/0309048>.
- Yifei Shen, Bo Li, and Xinjie Zhang. Skillopt-lite: Better and faster agent self-evolution via one line of vibe, 2026. URL <https://arxiv.org/abs/2607.03451>.

- Charles L. Wang, Keir Dorchen, and Peter Jin. On the statistical limits of self-improving agents, 2026a. URL <https://arxiv.org/abs/2510.04399>.
- Wenyi Wang, Piotr Piekos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025. URL <https://arxiv.org/abs/2510.21614>.
- Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents, 2026b. URL <https://arxiv.org/abs/2607.12227>.
- Hu Wei. Architectural design decisions in ai agent harnesses, 2026. URL <https://arxiv.org/abs/2604.18071>.
- Lilian Weng. Harness engineering for self-improvement. <https://lilianweng.github.io/posts/2026-07-04-harness/>, 2026.
- Zhaotian Weng, Antonis Antoniadis, Deepak Nathani, Zhen Zhang, Xiao Pu, and Xin Eric Wang. Group-evolving agents: Open-ended self-improvement via experience sharing, 2026. URL <https://arxiv.org/abs/2602.04837>.
- Wu et al. Automem: Automated learning of memory for llm agents, 2026. Cited for context on memory as a cognitive skill component; excluded from the harness evolution survey.
- Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-swe-agent: Can software engineering agents self-evolve on the fly?, 2025. URL <https://arxiv.org/abs/2511.13646>.
- Amy Xin, Jiening Siow, Junjie Wang, Zijun Yao, Fanjin Zhang, Jian Song, Lei Hou, and Juanzi Li. Eurekaagent: Agent environment engineering is all you need for autonomous scientific discovery, 2026. URL <https://arxiv.org/abs/2606.13662>.
- Tianshi Xu, Hui Feng Wen, and Meng Li. Adapting the interface, not the model: Runtime harness adaptation for deterministic llm agents, 2026. URL <https://arxiv.org/abs/2605.22166>.
- Xiyuan Yang, Jiaru Zou, Rui Pan, Ruizhong Qiu, Pan Lu, Shizhe Diao, Jindong Jiang, Hanghang Tong, Tong Zhang, Markus J. Buehler, Jingrui He, and James Zou. Recursive multi-agent systems, 2026a. URL <https://arxiv.org/abs/2604.25917>.
- Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. Skillopt: Executive strategy for self-evolving agent skills, 2026b. URL <https://arxiv.org/abs/2605.23904>.
- Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Guangxiang Zhao, Lin Sun, Xiangzheng Zhang, and Tong Yang. Harness-bench: Measuring harness effects across models in realistic agent workflows, 2026. URL <https://arxiv.org/abs/2605.27922>.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement, 2025. URL <https://arxiv.org/abs/2410.04444>.
- Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves, 2026a. URL <https://arxiv.org/abs/2606.09498>.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents, 2026b. URL <https://arxiv.org/abs/2505.22954>.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Hyperagents: Metacognitive self-improvement via editable meta-mechanisms, 2026c. URL <https://arxiv.org/abs/2603.19461>.

A HSI 智能体系统的实现细节

本节补充了主文中省略的 HSI 实现细节，旨在描述实现分层架构所需的执行环境、工具接口、内存组织以及作用域隔离机制。

A.1 智能体执行接口

所有 HSI 组件均使用相同的冻结大语言模型 M 和共享的 `react()` 执行原语进行实例化。每个作用域提供不同的工具集和系统上下文，而底层模型保持不变。

在每一步中，模型接收当前消息历史、可用工具和任务特定上下文，并通过以下循环产生动作：

$$a_t = M(o_t, \mathcal{T}, C_t), \quad (7)$$

其中 o_t 表示当前观测， $\mathcal{T}$ 为可用工具集， C_t 代表作用域特定上下文。所选动作可以修改文件、请求评估、记录信息或终止当前阶段。

三个作用域仅通过其工具可用性加以区分：

- **任务测试框架作用域**：与基准环境交互并执行当前测试框架 H 。
- **进化器作用域**：修改测试框架目录内的文件，并在开发任务上调用评估。
- **元进化器作用域**：修改进化策略目录 `evolution/` 内的文件。

这种分离确保了同一模型可以在不同的抽象层次上运行，而无需引入额外的学习参数或外部优化器模型。

A.2 进化工具接口

进化器作用域配备了一小组原子工具，允许 M 检查、修改和评估候选测试框架。

文件操作接口包括：

- 读取：检查现有源文件；
- 写入：创建新文件；
- 编辑：修改现有实现；
- 执行：运行用于调试或验证的 shell 命令。

除文件操作外，HSI 还提供演化专用原语：

- 规划：维护迭代局部推理笔记；
- `compact_context`：在上下文预算受限时总结先前的交互；
- 评估：执行当前测试框架并返回环境反馈；
- 经验教训：记录可复用的洞见，供后续迭代使用；
- `end_evolution`：终止当前进化过程。

该框架未规定这些操作之间的顺序。模型根据观察到的反馈和当前进化目标，自行决定何时以及如何使用每个工具。

A.3 记忆组织

HSI 维护多个具有不同持久性属性的记忆通道。

迭代局部记忆。在单次进化迭代期间，进化器维护一个存储在 plan.md 中的临时笔记本。该记忆存储中间假设、调试笔记和短期决策。若候选版本被丢弃，该记忆将随相应代码状态一并回滚。

持久进化记忆。在多次迭代中，HSI 维护一个持久化的经验档案，存储于 BOOTSTRAP.md 中。该档案中的条目总结了先前发现的模式、失败的方向以及可复用的演化指导。种子选择在生成未来迭代的假设时可以访问这些信息。

演化图记忆。累积演化图 G_t 存储已提交的框架版本及其关系。每个节点包含：

- 对应的框架快照；
- 获得的奖励；
- 描述演化步骤的元数据。

边编码版本之间的语义关系，例如扩展现有方法、修复失败模式或探索不同方向。该图为未来的种子选择提供长期的结构化记忆。

A.4 轨迹压缩与探测机制

将所有历史轨迹直接暴露给大语言模型（LLM）会超出可用的上下文预算，并引入不必要的噪声。因此，HSI 采用一种探针机制，从先前的执行历史中检索压缩摘要。

该探针作为一个辅助查询过程运行：

$$z = \text{probe}(\mathcal{T}_{\text{history}}, q), \quad (8)$$

其中 $\mathcal{T}_{\text{history}}$ 表示存储的轨迹， q 是当前智能体指定的查询。探针返回紧凑的摘要，而非原始交互轨迹。

例如，元进化器可以查询：

- 哪些种子选择行为与成功的迭代相关；
- 哪些进化模式经常导致回归；
- 哪些假设结构出现在大幅改进之前。

该机制使元进化器能够利用长期进化历史，同时保持其推理上下文的有界性。

A.5 作用域隔离与可编辑边界

HSI 在三个层级作用域之间强制执行明确的边界。

任务执行器 H 可由进化器编辑，但不能修改进化策略 Σ 。相反，元进化器可以修改 Σ ，但在元进化过程中不能直接更改任务执行器。任何在授权目录之外的修改都会被拒绝。

元进化器自身的执行逻辑从不可变的初始化模板加载。因此，递归修改过程终止于固定的外部边界：

$$M \rightarrow H \rightarrow \Sigma \rightarrow \text{frozen anchor}. \quad (9)$$

这种层级结构为受控的自我修改提供了所需的结构性约束：较低层保持可适应性，而最外层的执行边界保持固定。

A.6 评估接口

评估在可编辑表面之外执行。执行器通过固定接口接收任务：

`using_harness(agent, task).` (10)

尽管 H 的内部实现可能在不同迭代中发生变化，但该接口保持不变。因此，所有进化版本都在相同的任务注入机制下运行，并可使用相同的开发、验证和测试协议进行比较。

评估器同时返回标量奖励和可选的文本反馈。标量奖励用于候选方案比较，而文本反馈提供定性信息，可指导后续进化。

B 技术附录与补充材料

B.1 各套件实验配置

表 3 记录了本文中每个 BALROG 套件的实验配置。所有套件共享的配置如下。主干模型为 DeepSeek-V4-Flash，通过 `deepseek-v4-flash-preview` 应用程序编程接口服务访问。进化器作用域运行 $T = 5$ 次外层迭代，每次迭代最多 80 个 `react()` 步骤，LCB 奖励为 $z = 0.5$ ，启用思考，推理努力设为最大。任务-测试框架作用域在禁用思考且温度为 0 的条件下运行。元进化器作用域每次迭代最多 50 个 `react()` 步骤，采用贪心存档，种子选择和提交池均可进化，种子假设注入每次迭代的第一个系统提示，并在种子选择期间启用简短的种子验证探测（最多三次 `evaluate()` 调用）。终端最佳版本选择阶段是一个固定的、不可进化的智能体阶段，在每次进化结束时运行一次。初始测试框架不进行预评估；第 1 次迭代冷启动。

只有表 3 中的条目在各套件间有所不同。“设置”指 § 4.1 中定义的两协议（A：全量集内分布，B：子套件划分并保留测试集）。“开发比例”和“验证比例”是分配给开发集（进化奖励信号）和验证集（最佳版本选择）的套件比例。“测试回合数”是最终测试评估时每个任务的回合数。“开发回合数”是进化期间每个任务的回合数（更廉价的开发时估计，噪声由 LCB 奖励和最终测试测量吸收）。“测试重复次数”是进化后完整测试集重新评估的次数。“元”表示是否启用元进化器作用域。“提交最佳步骤数”是终端最佳版本选择阶段的步骤预算。

表 3：各套件实验配置。共享设置列于上文的叙述中；只有以下条目在各套件间有所不同。

Suite	Setup	Dev	Val	Test ep.	Dev ep.	Test rep.	Meta	Submit-best
TextWorld	A	1.0	0.00	10	3	3	off	50
BabyAI	A	1.0	0.00	10	3	3	on	80
Crafter	A	1.0	0.00	5	3	3	on	50
MiniHack	A	1.0	0.00	5	1	3	on	80
NLE	A	1.0	0.00	5	1	1	on	50
BabaIsAI-BreakStop	B	0.8	0.20	5	1	1	off	80
BabaIsAI-GoTo	B	0.8	0.25	5	1	1	on	80
BabaIsAI-Make	B	0.8	0.25	5	1	1	on	80

B.2 测试框架工程与智能体自进化方法的详细综述

我们提供了对 2025 至 2026 年间框架工程和智能体自进化领域 31 项代表性工作的扩展综述，按研究方向组织。AutoMem [吴等人, 2026 年] 被排除在外。

本综述聚焦于记忆作为认知技能组成部分，而非进化本身。

B.2.1 哥德尔式递归自我改进

哥德尔智能体 [尹等人, 2025年] (Yin 等人, 北京大学/加州大学圣塔芭芭拉分校, 2025 年)。首个受哥德尔机启发的基于大语言模型的自指智能体框架。通过递归主函数实现自指改进：自检（读取当前算法）、交互（评估性能）、自更新（通过大语言模型生成的猴子补丁修改代码）、持续改进（递归调用决策算法）。关键结果：受限版本（哥德尔基础，GPT-3.5）优于所有基线：DROP 80.9%，MGSM 64.2%，MMLU 70.9%，GPQA 34.9%；完整进化 $\approx \$15$ （对比元智能体搜索的 \$300）；在 24 点游戏中，智能体自发地从大语言模型方法切换到搜索算法，达到 100% 准确率。

达尔文哥德尔机 (Darwin Gödel Machine, DGM) [Zhang 等, 2026b] (Zhang 等, 不列颠哥伦比亚大学/向量研究所/Sakana AI, ICLR 2026)。首个实用的基于基础模型的哥德尔机，结合了自指代码修改与基于群体的开放式探索。初始化为单个轻量级编码智能体 (Claude 3.5 Sonnet, 仅具备 bash 和编辑工具)。经过 80 次迭代：从档案中按性能成正比、子代数量成反比选择父代；父代分析基准日志，提出新特性，修改自身代码库 (图灵完备的 Python)；分阶段评估 (10 $\rightarrow$ 50 $\rightarrow$ 200 个任务)。关键结果：SWE-bench 20.0% $\rightarrow$ 50.0%；Polyglot 14.2% $\rightarrow$ 30.7%（超越手工设计的 Aider）；贪心消融仅 39.7%，而完整 DGM 为 50.0%，证实基于档案的探索至关重要；自动发现细粒度文件编辑、多补丁排序、自动摘要以及历史感知的补丁生成；档案树揭示关键创新经由性能较低的中介节点触发。

赫胥黎哥德尔机 (Huxley-Gödel Machine, HGM) [Wang 等, 2025] (Wang 等, 阿卜杜拉国王科技大学, 2025)。识别并解决了基准性能与自我改进潜力之间的不匹配问题。引入分支级元生产力 (Clade-level Meta-Productivity, CMP)：聚合智能体整个分支中后代的性能，以衡量自我改进潜力。对分支成功率的贝塔分布使用汤普森采样进行节点扩展。定理 1 证明，在合理假设下，获得真实 CMP 足以模拟哥德尔机。关键结果：SWE-Verified-60 56.7%（对比 DGM 53.3%，SICA 50.0%）；Polyglot 30.5%（对比 DGM 27.1%）；比 DGM 快 $2.38 \times - 6.86 \times$ 倍；CMP 估计值与经验 CMP 的相关性为 0.778，而 DGM 为 0.285；智能体发现迭代细化和嵌套差异补丁结构；迁移至 GPT-5 在 SWE-Lite 上达到 57.0%，与最佳手工设计智能体相当。

群体进化智能体 (Group-Evolving Agents, GEA) [翁等人, 2026年] (Weng 等人, 加州大学圣塔芭芭拉分校, 2026 年)。将进化单元从个体转变为群体，并具有明确的群体内经验共享机制。在每次迭代中，通过性能-新颖性评分选择 $K=2$ 个父代；它们的进化轨迹（代码补丁、预测任务补丁、执行日志、评估结果）被聚合到一个共享池中；每个智能体的反思模块 (GPT-o1) 分析共享经验以生成进化指令。关键结果：SWE-bench Verified 达到 71.0%（对比 DGM 的 56.7%）；Polyglot 达到 88.3%（对比 DGM 的 68.3%）；最佳智能体整合了来自 17 个独特祖先（占种群 28.3%）的 8/9 项关键创新，而 DGM 仅从 9 个祖先（占 15.0%）整合了 5/9 项；最差的 GEA 前五智能体 (58.3%) 严格优于 DGM 的最佳智能体 (56.7%)；框架级缺陷在 1.4 次迭代内修复（对比 DGM 的 5.0 次）。

自改进编码智能体 (Self-Improving Coding Agent, SICA) [罗贝恩斯等人, 2025年] (Robeyns 等人, 布里斯托大学/iGent AI, 2025 年)。首个完全自指涉的编码智能体，消除了元智能体与目标智能体之间的区别：该智能体编辑自身的完整 Python 代码库。自改进循环：在基准上评估 $\rightarrow$ 最佳智能体晋升为元智能体 $\rightarrow$ 归档分析子智能体提出改进 $\rightarrow$ 软件开发子智能体实施 $\rightarrow$ 独立验证。关键结果：SWE-bench (50 个问题子集) 提升 17 个百分点 $\rightarrow$ 53% (+36, 15 次迭代)；智能体自主发明了智能编辑器、抽象语法树符号定位器、混合符号定位器、上下文敏感的差异最小化。关键负面结果：智能体框架饱和。在 AIME 2024/GPQA Diamond 推理任务上，脚手架相比原始模型没有带来增益（在 $\sim 76\%$ 处达到平台期，而裸 o3-mini-high 为 87%），这表明对于强推理模型，脚手架可能打断而非增强内部推理链。

超智能体 (HyperAgents) [Zhang 等, 2026c] (Zhang 等, 不列颠哥伦比亚大学/向量研究院/爱丁堡大学/纽约大学/元公司基础人工智能研究院/元超级智能实验室, 2026)。通过使元机制本身可编辑来扩展动态生成元机制 (DGM)。不同于动态生成元机制手工设计且固定的指令生成函数，任务智能体与

元代理被融合进一个单一的可编辑 Python 程序（超代理），因此生成新代理的过程本身成为自我修改的目标。关键结果：多语言（训练子集） $0.140 \rightarrow 0.340$ （置信区间 $0.300 - 0.380$ ），在没有编码特定手工设计的情况下匹配了 DGM 的编码性能；论文评审 $0.0 \rightarrow 0.710$ ，超过了开源的 AI-Scientist-v2 静态基线（ 0.630 ）；机器人奖励设计 $0.060 \rightarrow 0.372$ ，超过了直接奖励默认值（ 0.348 ）；从论文评审 + 机器人奖励到 IMO 评分的跨域迁移达到 $\text{imp}@50 = 0.630$ vs. DGM 定制 ≈ 0 ，而组合流水线（迁移 + 200 次迭代）达到 0.640 。两项消融（无自我改进；无开放式探索）在论文评审和机器人奖励设计上产生接近零的改进（ $p < 0.05$ ），证实两个组件都是必需的。定性发现：超代理自发演化出持久记忆和性能跟踪基础设施，这些是领域无关的元级能力，是涌现出来的而非被指定的。

递归智能体束（Recursive Agent Harnesses, RAH）[Lumer 等, 2026]（Lumer 等, 普华永道, 2026）。将完整的智能体束（文件系统工具、代码执行、规划）作为递归单元，而非单纯的模型调用。父智能体通过代码执行生成（使用 `asyncio.gather` 的 Python 脚本，绕过每轮 API 上限）或 JSON 工具调用生成来派生子智能体。关键结果：固定 GPT-5 主干，RAH 将 Codex 基线从 71.75% 提升至 81.36% （ $+9.61$ 个百分点）；sonnet-4.5 结合 RAH 达到 89.77% ；在 $1K - 4M$ 令牌上下文长度下均表现稳健。

EurekAgent [Xin 等, 2026]（Xin 等, 2026）。将“环境工程”确立为核心研究方向，从四个维度设计智能体环境：权限工程（Docker 隔离、隐藏评估器）、工件工程（文件系统 + Git 共享内存）、预算工程以及人在回路工程。关键结果：在三个数学任务和 TriMul CUDA 核函数工程上取得新的最先进水平（较最佳人类结果提升 4.3% ）；MLE-Bench Lite 任意奖牌率 85.71% ；发现 26 圆填充最先进方案，API 成本为 $\$ < 11$ 。

B.2.2 束工程与在线自适应

元束（Meta-Harness）[Lee 等, 2026b]（Lee 等, 斯坦福大学/KRAFTON/麻省理工学院, 2026）。开创性工作，将束优化形式化为外层代码空间搜索。更强的编码智能体（Claude Code + Opus-4.6）作为提议者，拥有对所有先前束候选源代码、评分和执行迹的完整文件系统访问权限，每次评估最多可获取 1000 万令牌的诊断信息，比先前的文本优化器（OPRO、TextGrad、OpenEvolve）大三个数量级。提议者通过 `grep`、`cat` 等工具选择性检查先前工件（每次迭代中位数 82 个文件，引用 > 20 个先前候选）。关键结果：在线文本分类上较 ACE 提升 $+7.7$ 个百分点，上下文令牌减少 $4\times$ ；在 5 个留出模型上 IMO 级数学推理平均提升 $+4.7$ 个百分点；TerminalBench-2 上 Haiku-4.5 智能体排名第一（ 37.6% ）。提议者发现了人类专家未明确设计的环境引导策略。

自动测试框架（AutoHarness）[Lou 等, 2026]（Lou 等, 谷歌深度思维, 2026）。该工作证明双子座 2.5 闪电模型（Gemini-2.5-Flash）能够通过汤普森采样引导的树搜索合成代码测试框架。两种框架模板：框架即动作验证器（采用带学习 `is_legal_action()` 的拒绝采样）与框架即策略（代码直接输出动作，无需大语言模型调用）。关键结果：在 145 个文本竞技场（TextArena）游戏上合法动作率达 100% （相比之下，卡格尔国际象棋中双子座模型因非法走子导致的失败占 78% ）；框架即策略取得 0.870 的平均奖励，超越更大规模模型。

人工智能智能体框架中的架构设计决策 [魏, 2026]（Wei, 2026 年）。一项基于协议的经验研究，涉及 70 个公开可用的智能体项目，识别出五个反复出现的设计维度（子智能体架构、上下文管理、工具系统、安全机制、编排）及其选项谱系。报告了共现关系（例如，执行隔离与结构化安全控制的提升度为 3.4 ）以及五种典型架构模式。

自测试框架（Self-Harness）[Zhang 等, 2026a]（Zhang 等, 上海人工智能实验室, 2026）。提出固定模型无需外部帮助即可改进自身运行框架，具体分为三个阶段：弱点挖掘（基于验证器锚定的失败签名对执行轨迹进行聚类）、框架提案（生成多样化的最小候选修改）以及提案验证（在保留集与留出集上采用非回归接受规则）。关键结果：MiniMax M2.5 相对提升 $40.5\% \rightarrow 61.9\%$ （ $+53\%$ ）；Qwen3.5-35B-A3B 提升 $23.8\% \rightarrow 38.1\%$ （ $+60\%$ ）；GLM-5 提升 $42.9\% \rightarrow 57.1\%$ （ $+33\%$ ）。定性分析揭示了模型特定的适应性：不同模型从相同的初始框架和算法出发，产生了完全不同的框架修改。

智能体框架工程 (Agentic Harness Engineering, AHE) [Lin 等, 2026a] (Lin 等, 复旦大学/北京大学, 2026)。论证框架演化的瓶颈在于可观测性, 而非智能体能力。三大支柱: 组件可观测性 (每个可编辑组件以文件形式暴露)、经验可观测性 (数百万原始轨迹标记被提炼为分层、可下钻的证据语料库) 以及决策可观测性 (每次编辑都附带一个自我声明的预测, 随后进行验证)。关键结果: 终端基准 2 (TerminalBench-2) 的首次通过率 (pass@1) 在 10 次迭代内从 69.7% 提升至 77.0%; 冻结的框架迁移到软件工程基准验证集 (SWE-bench-verified) 时取得最高综合成功率, 同时使用的标记数减少 12%。自我归因: 修复精确率为 33.7%, 修复召回率为 51.4% ($5\times$ 随机), 但回归精确率仅为 11.8% ($2\times$ 随机)。

HarnessX [Chen 等人, 2026b] (达尔文智能体团队, 2026)。三项创新: (1) 将组合式测试框架形式化为附加在生命周期钩子上的类型化处理器, 并配以九维分类法; (2) 基于符号适应与强化学习理论之间的“操作镜像”构建 AEGIS 进化引擎, 具备针对奖励黑客行为、灾难性遗忘和探索不足的防御机制; (3) 通过跨测试框架的 GRPO 实现测试框架与模型的协同进化。关键结果: 在 15 个模型-基准配置上平均提升 +14.5% (在 ALFWorld 上使用 Qwen3.5-9B 提升 +44.0%); 变体隔离解决了异构 GAIA 上的停滞问题 ($\Delta = 0.0 \rightarrow +13.6\%$); 协同进化相比仅使用测试框架额外提升 +4.7%。所有三种预测的强化学习病态现象均得到实证确认。

HarnessForge [Chen 等人, 2026a] (Chen 等人, 北京航空航天大学/清华大学, 2026)。将大语言模型智能体系统形式化为测试框架-策略对 $G = (H, R)$, 其中 $H = (P, A, M)$ 指定规划、行动和记忆组件。迭代交替进行故障引导的测试框架定制与测试框架条件化的策略对齐 (使用成功轨迹进行监督式迹对齐)。关键结果: 在 ToolHop/SearchQA/TMDB/API-Bank 上, 相比每个度量指标的最强基线平均提升 +3.56%; 兼容性矩阵显示, 最佳测试框架与不匹配的策略组合时性能下降, 证明仅靠测试框架工程是不够的。

TTHE [Nie 等人, 2026] (Nie 等人, 香港浸会大学/中国科学技术大学, 2026)。将可执行测试框架视为测试时自适应的状态, 维护一组候选测试框架, 并仅使用未标记的执行迹对其进行优化。关键结果: BIRD 12.0% $\rightarrow$ 50.0%; LiveCodeBench 30.0% $\rightarrow$ 38.3%; SWE-bench 20.0% $\rightarrow$ 35.0%; claw-eval 48.9% $\rightarrow$ 69.8% (+20.9 个百分点)。预言机分析揭示 $\sim 14pp$ 选择遗憾和 $\sim 30\%$ 的覆盖缺口, 将代理信号可靠性确定为核心挑战。

Live-SWE-Agent [Xia 等人, 2025] (Xia 等人, 伊利诺伊大学厄巴纳-香槟分校, 2025)。首个“活体”软件智能体, 在解决问题过程中通过创建、修改和使用自定义工具在运行时实现自我进化。从最小脚手架 (mini-SWE-agent, ~ 100 行代码, 仅 bash) 开始, 并在每次行动后添加反思步骤。关键结果: 使用 Gemini 3 Pro 在 SWE-bench Verified 上达到 77.4%, 优于所有已知智能体; 零离线成本, 每个任务额外开销 0.02 - 0.12 美元; 关键发现: GPT-5-Nano 在尝试自我进化时性能下降 (-68.2% (相对值)), 为自我进化确立了模型能力下限。

持续式框架 (Continual Harness) [Karten 等人, 2026] (Karten 等人, 普林斯顿大学/谷歌深度思维, 2026)。将框架演化扩展到免重置在线环境中的具身智能体。精炼器在单个连续回合中每 F 步编辑完整框架状态 (系统提示、子智能体、技能、记忆)。关键结果: 双子玩宝可梦 (Gemini Plays Pokemon), 首个完成多款宝可梦角色扮演游戏的人工智能; 从零开始的持续式框架恢复了与手工设计专家框架差距的大部分; 闪光精简版 (Flash-Lite) 停滞在 20% 以下, 表明存在能力下限。

自适应自动框架 (Adaptive Auto-Harness) [刘等人, 2026年] (Liu 等人, 埃默里大学/亚马逊, 2026年)。将框架构建形式化为遗憾最小化问题, 并分解为演化损失 L_{evo} 与适应损失。关键结果: PolyBench 准确率达 80.9% (对比元框架 (Meta-Harness) 的 50.8%); CTF-Dojo 的 Pass@1 为 50.2%; 适应损失在所有周期中保持正值, 证实单一固定框架对于流式异构工作负载永远不够。

生命周期框架 (LIFE-HARNESS) [徐等人, 2026 年] (Xu 等人, 北京大学, 2026 年)。将框架适应组织为四个生命周期层: 环境契约、程序技能、动作实现和轨迹调节。关键结果: 在 116/126 (92%) 的模型-环境设置中取得改进, 平均相对改进达 88.5%; 从 Qwen3-4B 演化出的框架可迁移至其他 17 个模型。

B.2.3 缰绳-模型协同演化

框架与解决方案协同演化系统 (HASE) [罗等人, 2026a] (Luo 等人, 香港大学/中国移动, 2026 年)。将解决方案生成与框架编辑置于统一的群体相对策略优化 (GRPO) 动作空间中, 使单一模型能够协同演化权重、框架和任务解决方案。区分了可自由编辑的引导组件与评估组件。

组件（仅在本地评估器与实际评估器出现分歧时修复）。关键结果：Qwen3-8B 在文本分类任务上与 GPT-OSS-120B 和 Claude Code 持平（86.98% 对 86.8%）；Alpha 因子挖掘的信息检索指标达到 1.70（对比 GPT-OSS-120B 的 0.80）；圆形装填问题中实现了渐进式评估器修复。

SIA [Hebbar 等人, 2026] (Hebbar 等人, Hexo Labs/牛津大学, 2026)。首个在单一闭环中同时更新框架与模型权重的系统。反馈智能体动态选择框架更新或权重更新（PPO/熵优势加权/GRPO）。关键结果：LawBench 70.1%（对比仅框架更新 50.0%）；TriMul CUDA 核函数较仅框架更新加速 $1,017 \mu s$, $14.02 \times$ 倍；框架更新与权重更新在互补的变更空间中运作。

技能优化器与轻量技能优化器 (SkillOpt & SkillOpt-Lite) [杨等人, 2026b, 沈等人, 2026] (Yang 等人, 微软; Shen 等人, 2026 年)。技能优化器 (SkillOpt) 将深度学习风格的控制引入文本空间技能优化（展开批次、文本学习率、验证门、被拒编辑缓冲区、慢速/元更新）。轻量技能优化器 (SkillOpt-Lite) 将其形式化为零阶优化，并遵循三个原则（共识挖掘、验证门控、基于文件系统的探索）。框架优化器 (HarnessOpt) 使具有优化框架的较小模型能够超越较大模型。

B.2.4 递归与多智能体框架系统

递归框架自改进 (RHI) [Lee et al., 2026a] (Lee 等, Sakana AI/加州大学伯克利分校, 2026)。将框架定义为提示级规范（角色、指令、契约、跳数），并利用轨迹局部自比较（仅与紧邻的前一个框架进行成对偏好反馈，而非与群体比较）迭代优化。关键结果：sonnet-4.6-high 经过 2 次 RHI 迭代后优于 sonnet-4.6-max；opus-4.8-high 结合 RHI 优于 opus-4.8-ultracode；推理成本最多降低 60%；收益源于改进的通信契约减少了跨组件冗余。

递归多智能体系统 (RecursiveMAS) [杨等人, 2026a] (Yang 等人, 斯坦福大学/伊利诺伊大学厄巴纳-香槟分校/英伟达/麻省理工学院, 2026 年)。通过潜在空间递归计算与轻量级 RecursiveLink 模块，将递归扩展从单一模型延伸至多智能体系统。关键结果：在 9 个基准上平均准确率提升 8.3%；推理加速 $1.2 \times - 2.4 \times$ 倍；令牌减少 34.6% - 75.6%；可训练参数仅 13.12M（占完整模型的 0.31%）。

B.2.5 评估、归因与理论基础

重新思考测试框架演进的评估 [Wang 等人, 2026b] (Wang 等人, 艾伦人工智能研究所/华盛顿大学, 2026 年)。对测试框架演进研究的关键性批判。在匹配反馈与推理预算的对照实验中显示：无单元测试时，测试框架演进为 67.4%，而并行采样为 72.3%；有单元测试时，分别为 75.8% 与 86.0%；在分离的搜索/评估任务上，泛化增益仅为 +0.6 个百分点。结论是大多数测试框架编辑“记忆修复而非提炼策略”，且剩余失败源于模型局限性而非测试框架缺陷。呼吁基准应满足两个条件：（1）任务足够困难且有改进空间；（2）性能高度依赖专用工具/技能/工作流。这直接启发了我们基于 BALROG 的实验设计。

测试框架更新并非测试框架收益 [Lin 等人, 2026b] (Lin 等人, 2026 年)。首次在 7 个大型语言模型 $\times$ 3 个基准上系统解耦测试框架更新与测试框架收益。关键发现：测试框架更新在基础能力上表现平稳（Qwen3.5-9B 与 Claude Opus 4.6 相当）；测试框架收益呈非单调性，中等层级模型获益最多（+19.3 个百分点），强层级模型触及天花板（+2.6pp），弱层级模型因技能加载率低（25.1% 对比 95.7%）和遵循率低（35.0% 对比 75.7%）而获益最少。这为我们的 NLE 负结果提供了直接的经验先例。

GSME（门控语义质量多样性）[罗等人, 2026b] (Luo 等人, EverMind AI, 2026 年)。建立了可信测试框架评估的黄金标准。三道门：有效性门（基础设施故障重新运行）、激活门（补丁仅在确实触发时记功）、显著性门（配对 2σ 检验， $z \geq 1.96$ ）。GSME 档案以（缺陷位置 $\times$ 缺陷原因）病理为键，而非以修复的任务为键，这是一种抗过拟合的归纳偏置。关键结果：在 7 个领域上密封测试增益为 +9 至 +15.5pp，保留了训练增益的 86 - 147%。非统计性的“均值提升”规则将 $\sim 60\%$ 的真正中性机制误记为有效。

Harness-Bench [Yao 等人, 2026] (Yao 等人, 北京大学, 2026)。首个以护栏机制为主要评估轴的诊断基准：包含 106 个沙盒离线任务、8 个工作流类别、6

个可配置线束 $\times$ 8 个模型后端。关键发现：最佳与最差线束之间存在 23.8 分的差距；更强的模型表现出更低的跨线束方差；五种反复出现的失效模式被映射到线束干预点。

SEAGym [Zheng 等人, 2026] (Zheng 等人, 清华大学, 2026)。强化学习风格的评估环境, 将兼容 Harbor 的基准转换为动态自进化任务源, 并提供多视角评估 (更新验证、分布内/分布外迁移、重放诊断、成本记录)。关键发现：批大小呈非单调性 (批大小 20 为最优)；源多样性对于从中间崩溃中恢复至关重要；有用的中间状态可能崩溃并在之后恢复, 仅凭最终分数无法察觉。

论自改进智能体的统计极限 [Wang 等人, 2026a] (Wang 等人, 哥伦比亚大学, 2025)。为自修改智能体建立了精确的“当且仅当”统计学习理论边界。核心结果：当且仅当策略可达假设族具有一致有界的 VC 维时, 自修改下无分布 PAC 保证得以保持 (定理 1)。双门护栏机制 (验证边际加容量上限) 通过标准 VC 速率预言不等式产生有限样本安全性 (定理 2)。识别出效用-学习张力：提升即时性能的效用驱动变化可能侵蚀可靠泛化的统计前提。在无界表示能力下, 存在合理的效用和 PAC 可学习问题, 使得证明触发的编辑使该问题在无分布意义下不可学习。五轴分解 (算法、表示、架构、基板、元认知) 将一切形式的自修改统一在单一容量准则之下。这为我们的实证发现提供了理论基础：护栏机制的进化无法超越模型能力边界——当任务所需函数复杂度超过模型可达 VC 维时, 任何护栏工程都无法弥合差距。

B.3 比较总结

表 4: 与 HSI 相比的代表性框架进化和自改进方法。提出者：谁提出框架编辑；表面：可编辑代码表面；领域：主要评估领域；特征：该方法的简短区分性贡献。

Method	Proposer	Surface	Domain	Feature
Meta-Harness	External stronger	Full harness code	Coding, math	Full-trajectory feedback
Self-Harness	Self (target)	Config interface	Coding	Model-specific edits
AHE	External stronger	Decoupled comps	Coding	Observability bottleneck
HarnessX	Multi-agent	Typed processors	5 benchmarks	Operational mirror to RL
DGM	Self	Full codebase	Coding	Archive-based search
HGM	Self	Full codebase	Coding	Clade meta-productivity
GEA	Self (group)	Codebase + exp	Coding	Shared experience pool
SICA	Self	Full codebase	Coding	Framework saturation
HyperAgents	Fused self	Codebase + meta	Coding, robots	Editable meta-mechanism
Live-SWE-Agent	Self (runtime)	Tools on-the-fly	Coding	Zero offline cost
TTHE	Self (test-time)	Harness popula- tion	Coding, SQL	Unlabeled trace adaptation
Rethinking Eval.	— (critique)	—	Coding	Test-time-scaling confound
Statistical Limits	— (theory)	—	PAC learning	VC bound $\iff$ learnability
HSI (Ours)	Same frozen M	3-layer hierar- chy	BALROG	Endogenous hierarchy with frozen outer anchor